

An empirical comparison and evaluation of minority oversampling techniques on a large number of imbalanced datasets

György Kovács*

Analytical Minds Ltd., Hungary
4933 Beregsurány, Árpád street 5

Abstract

Learning and mining from imbalanced datasets gained increased interest in recent years. One simple but efficient way to increase the performance of standard machine learning techniques on imbalanced datasets is the synthetic generation of minority samples. In this paper, a detailed, empirical comparison of 85 variants of minority oversampling techniques is presented and discussed involving 104 imbalanced datasets for evaluation. The goal of the work is to set a new baseline in the field, determine the oversampling principles leading to the best results under general circumstances, and also give guidance to practitioners on which techniques to use with certain types of datasets.

Keywords: imbalanced learning, SMOTE, minority oversampling, SMOTE variants

1. Introduction

In recent years, the topic of *imbalanced learning* gained remarkable interest among theorists and practitioners of machine learning and artificial intelligence. The primary focus of imbalanced learning is improving the performance of general purpose classifiers in the presence of some significant inter-class or intra-class variation in the number and/or distribution of training data samples [49]. See Figure 1 for an illustration of various types of imbalanced datasets. In practice, imbalanced learning problems are usually related to fields where the planned and/or cheap sampling of data is not feasible (for example, medical predictions [104, 2] and failure/anomaly detection [100, 24]). The reason why imbalanced problems need special treatment is that general machine learning techniques tend to overfit majority classes and densely sampled regions [49].

Literature distinguishes three main approaches to tackle the issues of imbalanced data [110]. *Cost sensitive techniques* [82, 70, 59] assign a higher cost to the misclassification of minority samples during the training process, in this way putting more emphasis on the generalization of minority samples (see Figure 2(a) for an illustration). *Classifier specific solutions* try to tweak the operation of classifiers to take into consideration the imbalanced nature of training data. These techniques are usually reasonable but heuristic adjustments made to the decision rules in instance-based methods [68, 110, 60] (see Figure 2(b) for an illustration). *Oversampling techniques* resolve the issue of imbalanced data by generating additional training

samples for the minority classes. One of the first oversampling techniques was Synthetic Minority Oversampling Technique (SMOTE) [20], generating minority samples by randomly sampling the line segments between neighboring minority instances (see Figure 2(c) for an illustration). Notable methodological overviews and comparative evaluations of imbalanced learning techniques are available in [21, 83, 71, 72, 50], in this study we focus on oversampling techniques.

As summarized in the recent overview [39], oversampling techniques are highly successful in various types of machine learning problems, from online learning to regression. The success of oversampling can be attributed to two main factors. On the one hand, unlike cost-sensitive and classifier-specific solutions, oversampling is a preprocessing step, thus, any pipeline of machine learning methods can be applied to the oversampled dataset. On the other hand, oversampling tries to tackle the root of imbalanced problems: the lack of data. Minority oversampling can be considered as a generalization of *data augmentation* [34] widely used in the image domain: slight geometrical transformations are supposed to leave the meaningful content of images intact, thus, the training set can be extended by generating geometrically distorted images to improve the generalization of the learning process.

Since SMOTE was published in 2003, more than 100 variants have been proposed [39]. Recent techniques are usually compared to conventional ones like the original SMOTE, borderline-SMOTE [47] and ADASYN [48]. Unfortunately, the evaluation methodology varies, thus, an unambiguous ranking of oversamplers based on the available comparisons cannot be deduced. Although some studies [83, 71, 45] compare and evaluate the already men-

*Corresponding author

Email address: gyuriofkovacs@gmail.com (György Kovács)

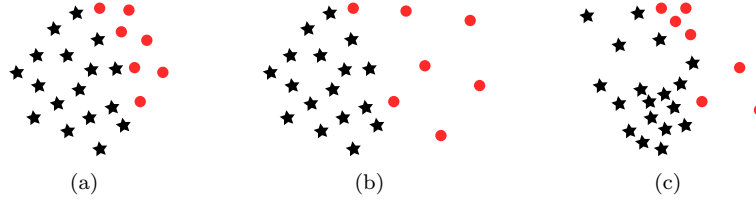


Figure 1: Various types of imbalanced data: imbalance in the number of instances (a); imbalance in the number and in the density of instances (b); imbalance in the number and locally varying imbalance in the density of samples (c).

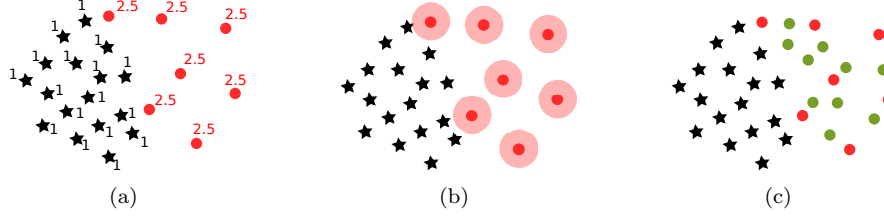


Figure 2: Cost sensitive techniques apply higher cost to the misclassification of minority instances - a possible cost scheme making the loss function balanced is indicated next to the samples (a); the classifier specific technique [68] treats the minority points as objects with some volume to increase the minority region - a possible choice of volumes is indicated by pink regions (b); oversamplers generate additional minority instances by local interpolation - the generated samples are indicated by green color (c).

tioned conventional techniques, for dozens of oversampling approaches comparable evaluations are not available, which necessitates a thorough evaluation and comparison to facilitate applications and further research. Finally, we emphasize that the proper evaluation of oversamplers is also pulled back by the lack of open source implementations: the most notable packages `imblearn` [63] (Python) and `smotefamily` (R) implement only 8 oversampling techniques together.

In this paper, we carry out a thorough comparison and evaluation of 85 variants of minority oversampling involving 104 imbalanced datasets. We emphasize that this work does not intend to be a methodological review of oversampling techniques, due to page limitations and the large number of algorithms covered, we cannot discuss all techniques and operating principles in details. The contributions to the field and benefits for theorists and practitioners are summarized as follows:

1. We set a new baseline for the research of oversampling methods. Given a ranking based on thorough testing, newly proposed oversampling methods are enough to be compared to the best performers to provide meaningful insight into their performance.
2. The analysis of the performance of various operating principles of oversampling techniques can aid and boost the development of new techniques based on the most successful principles.
3. Scientists and engineers working with real imbalanced data gain an insight into the performance of the various techniques on datasets with various characteristics, enabling them to select the most appropriate oversampling technique for the data they are working with.

priate oversampling technique for the data they are working with.

4. For reproducible science and the benefit of the community, all implementations and numerical results are published in the GitHub repository http://github.com/gykovacs/smote_variants.

The paper is organized as follows. In Section 2 the main characteristics of oversampling techniques, the databases used for testing, and the evaluation methodology are described. In Section 3 the test results are presented and evaluated, and finally, conclusions are drawn in Section 4.

2. Materials and methods

First, we introduce the notations used in the rest of the paper. The size of imbalanced datasets is denoted by $N \in \mathbb{Z}^+$ with $N_+, N_- \in \mathbb{Z}^+, N_+ + N_- = N$ minority (positive) and majority (negative) samples, respectively. The terms minority (majority) points/samples/instances are used interchangeably. In order to make the discussions compatible, we use the same abbreviations as [39] to refer to oversampling techniques.

2.1. The methods involved in the study

Although SMOTE has more than 100 variants in the literature [39], some techniques have not been involved in this study. Representatives of these are developed for online learning [35], work only with Support Vector Machines [16], binary or discrete features [112, 27], are not stable enough to be used in an automated framework [32], and we also found that some techniques are essentially the same

(like [107] and [106]), in these cases only one of them was considered.

For the ease of discussion and comparison, we have categorized the oversamplers based on their most characteristic operating principles. Although the categories are not disjoint, the overall performance of techniques in a particular category is expected to give some insight into how efficient the operating principle represented by the category is. We note that some techniques implement so unique approaches that none of the categories we defined fits them. In the rest of the subsection, we give the definitions of the categories, while the list of all techniques and category assignments is given in Table 1.

Ordinary sampling: Methods in this category implement the same sampling as SMOTE, based on the assumption that points along the line segments connecting neighboring minority instances belong to the minority class. Given two instances $\mathbf{x}^i, \mathbf{x}^j \in \mathbb{R}^d$, new samples are generated by

$$\mathbf{x}^n = \mathbf{x}^i + r \cdot (\mathbf{x}^j - \mathbf{x}^i), \quad (1)$$

where r is a uniformly distributed random number from $[0, 1]$.

Component-wise sampling: Methods in this category carry out the sampling of attributes independently. The assumption behind component-wise sampling is that the entire volume of a hypercube spanned by two neighboring minority samples belongs to the minority class.

Sampling by cloning: Oversampling can be done with no assumptions on the distribution of data by cloning minority instances. It is worth noting that cloning minority instances n times has a similar effect as assigning the integer cost $n + 1$ to the misclassification of that instance. Thus, classification after cloning is closely related to using a cost-sensitive variant of the classifier with the misclassification cost N/N_+ assigned to minority samples.

Borderline: These techniques implement some explicit steps to identify minority instances near the decision boundary and generate new samples in the neighborhoods of these borderline instances.

Using a sampling density: These techniques assign some importance scores to minority instances and the normalization of these scores results the sampling density $p_i, i = 1, \dots, N_+$ driving the sampling process. Particularly, p_i indicates the proportion of new instances to be generated in the neighborhood of the i th minority instance. Although ordinary SMOTE ($p_i = 1/N_+$) and borderline SMOTE ($p_i = 1/N_b$ for borderline samples, N_b denoting their number) can be considered as special cases of using a sampling density, only those techniques are included in this category which use some advanced importance scores. For example, multiple techniques use the number of majority samples in the neighborhoods of minority samples as the importance score: a large number of majority neighbors indicates that many additional minority samples need to be generated in the neighborhood to facilitate the correct classification of the minority sample.

Using density estimation: Methods in this category implement some (kernel-)density estimation for the minority class, and sample new instances from this density. It is important to distinguish density estimation and the use of sampling density. Density estimation based techniques sample the overall region of the minority class and can be expected to generate fewer samples near the decision boundary, as the density of the minority class decreases there. On the contrary, sampling density based techniques sample regions which are conceptually declared to be important by the importance score, and the common choice of importance scores makes them likely to generate samples near the decision boundary.

Dimensionality reduction: Techniques in this category use dimensionality reduction to map high dimensional data to lower dimensional spaces. Commonly implemented techniques are feature selection by cross-validation, principal component analysis (PCA) and t-distributed stochastic neighborhood embedding.

Use of classifiers: Some techniques use supervised classification at certain points of the oversampling process, for example, identify noisy samples based on the predicted positive-class probabilities, or implement memetic optimization and drive the optimization process by cross-validation scores.

Use of clustering: Techniques of this category use clustering methods to identify minority concepts, and then do the oversampling within the individual clusters independently.

Memetic: There are numerous methods using memetic algorithms to change the minority and/or majority sets, making the entire training set more suitable for supervised learning. These techniques usually use some classifier and drive the optimization process by cross-validation scores.

Application specific: Methods in this category were developed for specific applications, and evaluating their performance can give an insight into how useful application specific solutions are in general settings.

2.2. Implementation details and directives

In order to carry out the proposed evaluation and comparison of oversampling techniques, we have implemented all algorithms listed in Table 1. During the implementation, we tried to stick to the pseudo-codes in the papers, using similar parameter and variable names to enable the easy validation of the implementations. Additionally, we followed some general directives to resolve ambiguous steps, improve comparability and increase efficiency. These directives are summarized in the rest of this subsection.

The proportion parameter: As a matter of fact, the number of minority samples generated affects the subsequent classification performance. Some techniques let the user specify the number of samples to be generated, others (just like ordinary SMOTE) provide a coarse control on the number of instances to be generated (for example, generat-

ing H number of samples in the neighborhood of each minority instance), while memetic oversamplers usually generate an unpredictable number of samples. In order to standardize the algorithms in this respect, in most cases we managed to introduce a parameter called *proportion* referring to the proportion of the difference $N_d = (N_- - N_+)$ to be generated. For example, setting the proportion parameter to $p = 1$ means that $p \cdot N_d = N_d$ minority samples are generated and the dataset gets balanced. During the evaluation process, the proportion parameter is varied and the best subsequent classification results are used for comparison, eliminating the effect of the number of generated samples on the results.

Unspecified details: In some papers minor algorithmic details or parameter settings are not discussed in details, for example the parameters of the bat-optimization in [66] and the autoencoder network structure in [9]. We tried to resolve all ambiguous steps in the most meaningful way.

Reasonable parameters: For each parameter of each technique, we have identified a set of reasonable values, for example, for the proportion parameter $p \in \{0.1, 0.25, 0.5, 0.75, 1\}$. These sets enable the construction of meaningful parameter combinations for model selection.

Optimization: We found that some techniques have the time complexity $O(n^x)$, $x \geq 3$, making the evaluation unfeasible on datasets having more than a couple of hundred elements (for example [76]). In these cases, compromises were made and some accuracy was sacrificed for tractable time complexity.

The use of classifiers: Oversampling techniques are evaluated in subsequent classification scenarios. As mentioned before, memetic optimization based techniques [19, 66] use classifiers and cross-validation scores to drive the optimization of the oversampling process. Passing the subsequent classifier to these techniques would let them optimize sample generation for the subsequent classification task. The drawback of doing so is that cross-validation driven memetic search is extremely expensive, and optimizing oversampling for a given classifier would necessitate repeating the memetic optimization whenever the parameters of the classifier are changed. In order to keep the oversampling step separated from the classification step and keep the model selection procedure tractable in terms of runtime, our implementation uses the classifiers reported in the corresponding papers as default, instead of optimizing sample generation for subsequent classification steps.

Reproducing machine learning techniques is always prone to errors, in spite of all efforts, it is possible that our interpretations and/or implementations are not completely aligned with the intentions of the authors. In order to prevent well-established methods from being declared as worst performers due to mismatching interpretations or implementations, we will report only the best performing techniques in each evaluation scenario. The major changes and adjustments applied to the algorithms are documented in the source codes at http://github.com/gkovacs/smote_variants.

2.3. Datasets used for evaluation

Although the evaluation methodology of imbalanced learning techniques has not been standardized, the datasets used for evaluation has distilled to a commonly accepted set. Recent evaluations [71, 72] use datasets collected in the *Knowledge Extraction based on Evolutionary Learning* (KEEL) repository [3], most of which are based on multiclass datasets available in the UCI Machine Learning Repository [29], restructured as imbalanced binary classification problems by selecting some small cardinality classes to constitute the minority class and considering the union of all other classes as the majority class. This restructuring is reflected in the names of the datasets, for example, "ecoli-0-6-7.vs.3-5" refers to the UCI database "ecoli" restructured by selecting class labels 0, 6 and 7 to be the negative (majority) class and using class labels 3 and 5 as the positive (minority) class. Additionally to the KEEL datasets, we have included the imbalanced databases commonly used for the evaluation of variants of the k Nearest Neighbors (kNN) classifier [110, 109, 69, 68]. We recognized that there is a minor overlap, the datasets *vehicle* and *glass* in [110] are the same as datasets *vehicle0* and *glass2* in [3]. We also recognized that there is no difference between the *new-thyroid1* and *new-thyroid2* datasets of [3], and used only *new-thyroid1* for evaluation. Finally, we have excluded some the datasets: *ADA*, *HIVA*, *sylva*, *satimage*, *dermathology.6* and *page_blocks0* were found to be too large in terms of records and/or attributes to be included in the study, as applying the oversamplers of highest time complexities to these datasets is intractable. An overview of the most characteristic properties of the remaining 104 datasets is given in Table 2, sorted by the imbalance ratio

$$IR = \frac{N_-}{N_+}. \quad (2)$$

The datasets involved in the study show various characteristics, containing real, nominal, categorical and binary attributes, as well. Although some of the techniques cover the case of categorical attributes and propose some special treatment to handle them, most of the oversampling methods are inherently related to the features of the Euclidean space and the Euclidean distance. In order to make all techniques applicable to all datasets, we have applied some feature encoding steps to make the feature representation compatible with operations in the Euclidean space: (a) attributes with 1 unique value have been removed; (b) binary attributes were kept and handled as floating point values; (c) nominal and categorical attributes were one-hot encoded when they had less than 5 distinct values, otherwise label encoding was applied to keep the number of attributes relatively low compared to the size of the datasets, thus, avoiding the introduction of sparsity and changing the nature of datasets remarkably.

2.4. Performance measures

Although binary classification has widely accepted and standard measures to characterize the performance of clas-

Table 1: The oversampling methods being compared and the category assignments characterizing their main operating principles

method	noise removal	dimension reduction	uses classifier	componentwise sampling	ordinary sampling	sampling by cloning	memetic	density estimation	density based	changes majority	uses clustering	borderline	application	method	noise removal	dimension reduction	uses classifier	componentwise sampling	ordinary sampling	sampling by cloning	memetic	density estimation	density based	changes majority	uses clustering	borderline	application
1 SMOTE (2002)[20]				×										44 SMOTE-OUT (2014)[56]													
2 SMOTE-TomekLinks (2004)[8]	×								×					45 SMOTE-Cosine (2014)[56]													
3 SMOTE-ENN (2004)[8]	×								×					46 Selected-SMOTE (2014)[56]				×									
4 Borderline-SMOTE1 (2005)[47]											×			47 MWMOTE (2014)[7]											×	×	
5 Borderline-SMOTE2 (2005)[47]												×		48 PDFOS (2014)[41]								×					
6 AHC (2006)[25]									×	×	×			49 IPADE-ID (2014)[73]				×				×		×			
7 LLE-SMOTE (2006)[98]		×												50 RWO-sampling (2014)[106]													
8 cluster-SMOTE (2006)[24]										×				51 NEATER (2014)[4]									×	×			
9 distance-SMOTE (2007)[14]						×								52 SDSMOTE (2014)[67]					×						×		
10 ADASYN (2008)[48]						×			×		×			53 DSMOTE (2014)[76]									×				
11 SMMO (2008)[15]			×									×		54 G-SMOTE (2014)[91]				×									
12 polynom-fit-SMOTE (2008)[43]														55 NT-SMOTE (2014)[102]												×	
13 Stefanowski (2008)[94]	×					×			×		×			56 SSO (2014)[88]				×				×	×	×			
14 ADOMS (2008)[96]		×												57 Supervised-SMOTE (2014)[51]				×	×							×	
15 Safe-Level-SMOTE (2009)[11]					×						×			58 DEAGO (2015)[9]								×			×		
16 MSMOTE (2009)[52]		×										×		59 Gazzah (2015)[44]				×					×				
17 ISOMAP-Hybrid (2009)[46]		×	×						×					60 MCT (2015)[54]						×							
18 DE-oversampling (2010)[22]									×	×				61 ADG (2015)[80]										×			
19 CE-SMOTE (2010)[23]							×				×	×		62 SMOTE-IPF (2015)[89]				×					×				
20 Edge-Det-SMOTE (2010)[55]									×	×				63 KernelADASYN (2015)[95]								×			×		
21 SMOBD (2011)[17]		×							×	×				64 MOT2LD (2015)[101]					×					×			
22 SUNDO (2011)[18]												×		65 V-SYNTH (2015)[103]					×								
23 MSYN (2011)[36]														66 Lee (2015)[62]					×								
24 LN-SMOTE (2011)[75]														67 SPY (2015)[26]									×				
25 CBSO (2011)[5]						×			×	×				68 SMOTE-PSOBAT (2015)[66]					×	×				×			
26 E-SMOTE (2011)[28]		×					×			×				69 OUPS (2016)[87]					×								
27 Random-SMOTE (2011)[30]					×									70 SMOTE-D (2016)[97]													
28 NDO-sampling (2011)[108]							×					×		71 MDO (2016)[1]				×									
29 DSRBF (2011)[40]					×	×	×							72 VIS-RST (2016)[10]				×									
30 SVM-balance (2012)[37]			×						×					73 GASMOTE (2016)[53]					×	×							
31 TRIM-SMOTE (2012)[81]										×				74 A-SUWO (2016)[79]				×					×	×			
32 SMOTE-RSB (2012)[84]						×								75 SMOTE-FRST-2T (2016)[85]				×						×		×	
33 DBSMOTE (2012)[12]			×						×	×				76 AND-SMOTE (2016)[105]					×								
34 ASMOBD (2012)[99]			×								×			77 SMOTE-PSO (2017)[19]				×			×						
35 SN-SMOTE (2012)[42]						×								78 CURE-SMOTE (2017)[74]										×			
36 ProWSyn (2013)[6]							×							79 SOMO (2017)[31]										×			
37 SL-graph-SMOTE (2013)[13]												×		80 NRAS (2017)[86]				×									
38 NRSBoundary-SMOTE (2013)[38]												×		81 Gaussian-SMOTE (2017)[61]													
39 LVQ-SMOTE (2013)[78]												×		82 CCR (2017)[57]													
40 SOI-CJ (2013)[90]						×				×				83 ANS (2017)[93]					×				×				
41 Assembled-SMOTE (2013)[111]							×			×	×			84 AMSCO (2018)[65]				×			×		×				
42 ISMOTE (2013)[64]										×				85 kmeans-SMOTE (2018)[33]										×			
43 ROSE (2014)[77]						×																					

Table 2: Overview of imbalanced datasets involved in the study, d denoting the number of attributes

name	N	N_+	d	IR	name	N	N_+	d	IR
1 glass1[3]	214	76	9	1.82	53 PC1[110]	1109	77	21	13.40
2 ecoli-0_vs_1[3]	220	77	7	1.86	54 shuttle-c0-vs-c4[3]	1829	123	9	13.87
3 wisconsin[3]	683	239	9	1.86	55 yeast-1_vs_7[3]	459	30	7	14.30
4 pima[3]	768	268	8	1.87	56 glass4[3]	214	13	9	15.46
5 iris0[3]	150	50	4	2.00	57 ecoli4[3]	336	20	7	15.80
6 glass0[3]	214	70	9	2.06	58 page-blocks-1-3_vs_4[3]	472	28	10	15.86
7 german[110]	1000	300	29	2.33	59 abalone9-18[3]	731	42	8	16.40
8 yeast1[3]	1484	429	10	2.46	60 zoo-3[3]	101	5	16	19.20
9 haberman[3]	306	81	3	2.78	61 glass-0-1-6_vs_5[3]	184	9	9	19.44
10 vehicle2[3]	846	218	18	2.88	62 hypothyroid[110]	3163	151	25	19.95
11 vehicle1[3]	846	217	18	2.90	63 shuttle-c2-vs-c4[3]	129	6	9	20.50
12 vehicle3[3]	846	212	18	2.99	64 shuttle-6_vs_2-3[3]	230	10	9	22.00
13 glass-0-1-2-3_vs_4-5-6[3]	214	51	9	3.20	65 yeast-1-4-5-8_vs_7[3]	693	30	10	22.10
14 vehicle0[3]	846	199	18	3.25	66 glass5[3]	214	9	9	22.78
15 ecoli1[3]	336	77	7	3.36	67 yeast-2_vs_8[3]	482	20	10	23.10
16 hepatitis[110]	155	32	19	3.84	68 lymphography-normal-fibrosis[3]	148	6	23	23.67
17 SPECT_F[110]	267	55	44	3.85	69 flare-F[3]	1066	43	11	23.79
18 new.thyroid1[3]	215	35	5	5.14	70 car_good[3]	1728	69	6	24.04
19 ecoli2[3]	336	52	7	5.46	71 car-vgood[3]	1728	65	6	25.58
20 KC1[110]	2109	326	21	5.47	72 kr-vs-k-zero-one_vs_draw[3]	2901	105	6	26.63
21 segment0[3]	2308	329	23	6.02	73 kr-vs-k-one_vs_fifteen[3]	2244	78	6	27.77
22 glass6[3]	214	29	9	6.38	74 yeast4[3]	1484	51	10	28.10
23 yeast3[3]	1484	163	10	8.10	75 winequality-red-4[3]	1599	53	11	29.17
24 ecoli3[3]	336	35	7	8.60	76 poker-9_vs_7[3]	244	8	25	29.50
25 ecoli-0-3-4_vs_5[3]	200	20	7	9.00	77 kddcup-guess_passwd_vs_satan[3]	1642	53	38	29.98
26 yeast-2_vs_4[3]	514	51	8	9.08	78 yeast-1-2-8-9_vs_7[3]	947	30	10	30.57
27 ecoli-0-6-7_vs_3-5[3]	222	22	7	9.09	79 abalone-3_vs_11[3]	502	15	8	32.47
28 ecoli-0-2-3-4_vs_5[3]	202	20	7	9.10	80 winequality-white-9_vs_4[3]	168	5	11	32.60
29 glass-0-1-5_vs_2[3]	172	17	9	9.12	81 yeast5[3]	1484	44	10	32.73
30 yeast-0-3-5-9_vs_7-8[3]	506	50	10	9.12	82 kr-vs-k-three_vs_eleven[3]	2935	81	6	35.23
31 yeast-0-2-5-6_vs_3-7-8-9[3]	1004	99	10	9.14	83 winequality-red-8_vs_6[3]	656	18	11	35.44
32 yeast-0-2-5-7-9_vs_3-6-8[3]	1004	99	10	9.14	84 ecoli-0-1-3-7_vs_2-6[3]	281	7	7	39.14
33 ecoli-0-4-6_vs_5[3]	203	20	6	9.15	85 abalone-17_vs_7-8-9-10[3]	2338	58	8	39.31
34 CM1[110]	498	49	23	9.16	86 abalone-21_vs_8[3]	581	14	8	40.50
35 ecoli-0-1_vs_2-3-5[3]	244	24	7	9.17	87 yeast6[3]	1484	35	10	41.40
36 ecoli-0-2-6-7_vs_3-5[3]	224	22	7	9.18	88 winequality-white-3_vs_7[3]	900	20	11	44.00
37 glass-0-4_vs_5[3]	92	9	9	9.22	89 winequality-red-8_vs_6-7[3]	855	18	11	46.50
38 ecoli-0-3-4-6_vs_5[3]	205	20	7	9.25	90 kddcup-land_vs_portsweep[3]	1061	21	40	49.52
39 ecoli-0-3-4-7_vs_5-6[3]	257	25	7	9.28	91 abalone-19_vs_10-11-12-13[3]	1622	32	8	49.69
40 yeast-0-5-6-7-9_vs_4[3]	528	51	10	9.35	92 kr-vs-k-zero_vs_eight[3]	1460	27	6	53.07
41 vowel0[3]	988	90	13	9.98	93 winequality-white-3-9_vs_5[3]	1482	25	11	58.28
42 ecoli-0-6-7_vs_5[3]	220	20	6	10.00	94 poker-8-9_vs_6[3]	1485	25	25	58.40
43 glass-0-1-6_vs_2[3]	192	17	9	10.29	95 shuttle-2_vs_5[3]	3316	49	9	66.67
44 ecoli-0-1-4-7_vs_2-3-5-6[3]	336	29	7	10.59	96 winequality-red-3_vs_5[3]	691	10	11	68.10
45 led7digit-0-2-4-6-7-8-9_vs_1[3]	443	37	7	10.97	97 abalone-20_vs_8-9-10[3]	1916	26	8	72.69
46 ecoli-0-1_vs_5[3]	240	20	6	11.00	98 kddcup-buffer_overflow_vs_back[3]	2233	30	31	73.43
47 glass-0-6_vs_5[3]	108	9	9	11.00	99 kddcup-land_vs_satan[3]	1610	21	30	75.67
48 glass-0-1-4-6_vs_2[3]	205	17	9	11.06	100 kr-vs-k-zero_vs_fifteen[3]	2193	27	6	80.22
49 glass2[3]	214	17	9	11.59	101 poker-8-9_vs_5[3]	2075	25	25	82.00
50 ecoli-0-1-4-7_vs_5-6[3]	332	25	6	12.28	102 poker-8_vs_6[3]	1477	17	25	85.88
51 cleveland-0_vs_4[3]	177	13	23	12.62	103 kddcup-rootkit-imap_vs_back[3]	2225	22	47	100.14
52 ecoli-0-1-4-6_vs_5[3]	280	20	6	13.00	104 abalone19[3]	4174	32	8	129.44

sifiers, many of these are not suitable in imbalanced scenarios, since the performance on the majority class will be overrepresented. In imbalanced learning, the main goal is to improve the classification of minority (positive) samples, while maintaining a reasonable performance regarding the majority ones. Accordingly, based on the previous comparative studies [71, 72, 83], 4 measures have been selected to compare classification performance. Introducing the notations TP, TN, FP and FN for the number of true positive, true negative, false positive and false negative samples, respectively, and $P = TP + FN$, $N = TN + FP$, the selected measures are defined as follows.

G-score: the geometric mean of accuracies achieved on minority and majority instances:

$$G = \sqrt{\frac{TP}{P} \cdot \frac{TN}{N}}. \quad (3)$$

F₁-score: the harmonic mean of precision (PR) and recall (RE):

$$F_1 = 2 \cdot \frac{PR \cdot RE}{PR + RE}, \quad PR = \frac{TP}{TP + FP}, \quad RE = \frac{TP}{TP + FN}, \quad (4)$$

where precision measures the proportion of samples correctly classified as positive, and recall characterizes the proportion of all positive samples classified as positive.

AUC-score: (Area Under the receiver operating characteristic Curve) characterizes the area under the curve of sensitivities plotted against corresponding false positive rates

$$FPR = \frac{FP}{FP + TN} \quad (5)$$

at various probability threshold settings. According to one of the most intuitive interpretations, AUC is the probability that the classifier will assign a higher positive-class probability to a randomly selected positive sample than to a randomly selected negative sample.

Precision at Top 20% (P20): the study [83] on imbalanced evaluation measures proposed the use of the *precision at the top 20* measure, which is the percentage of truly positive samples among the 20 test records having the highest predicted probability of being positive. As some of the imbalanced datasets do not have 20 minority samples, we have changed the measure to the percentage of truly positive samples among the top 20% of test records ranked by the predicted positive-class probabilities.

2.5. Evaluation methodology

As oversampling techniques are evaluated in subsequent classification scenarios, the results highly depend on the classification technique used. For example, instance-based techniques are likely to work well with oversamplers equalizing the density of positive and negative samples near the decision boundary, on the contrary, neural networks might work better with oversamplers increasing the cardinality of the minority class to that of the majority class by sampling the entire minority manifold, since the classes will

be represented equally in the gradient of the loss function during backpropagation. In order to see which oversamplers work well with various types of classifiers, we have selected k Nearest Neighbors (kNN) [92], Support Vector Machines (SVM) [92], decision trees (DT) [92] and multi-layer perceptrons (MLP) [92] to be included in the study. In the rest of the subsection, the details of the evaluation methodology are described.

Parameter combinations for oversamplers: Based on the sets of reasonable parameter values identified for each oversampler, we have randomly chosen up to 35 parameter combinations for model selection (some techniques have less 35 parameter combinations altogether). The maximum number of parameter combinations is an empirical compromise made for tractable runtimes.

Parameter combinations for classifiers: For each classifier we have selected 6 different parameterizations for evaluation: linear SVM with $C \in \{1, 10\}$ (high C values improve SVM on imbalanced data [37]), L_1 and L_2 penalties with compatible hinge or squared hinge loss; kNN with $k \in \{3, 5, 7\}$, standard or distance weighted decision functions with L_2 distance; DT with Gini-impurity or entropy as splitting criterion, maximum depth of 3, 5 and unbounded; MLP with 1 hidden layer, RELU or logistic activation functions and hidden units specified as 10%, 50% and 100% of the number of input features.

Cross-validation: Classification performance is evaluated by repeated stratified k-fold cross-validation with 5 splits and 3 repeats. The number of splits was implied by the lowest number of minority samples in dataset *zoo-3*, the number of repeats is a compromise made for tractable runtimes. We note that the number of repeats is higher than the ones used in many recent studies (for example, [31] and [9] use 1 and 2 repeats, respectively), thus, more reliable results can be expected.

Evaluation: The evaluation is carried out by the usual cross-validation methodology involving all oversamplers with up to 35 random parameter combinations and all types of classifiers with 6 parameter combinations. The training set of each dataset fold is oversampled before classifier training. For comparable results, special emphasis is put on folding the datasets the same way in each test case. In each cross-validation scenario, the G, AUC, F1 and P20 scores were determined. In order to implement model selection, for each combination of dataset, oversampler type, and classifier type the highest scores were determined, and these scores are analyzed in the next subsection. In order to illustrate the volume of the study, we mention that the overall number of both the oversampling, training and prediction jobs is over 76 million.

3. Results

Due to page limitations, it is infeasible to analyze and discuss all aspects of oversampling techniques. In this section, we keep focusing on the most interesting theoretical and practical questions: Which techniques give the best

performance in general? How does the performance depend on the characteristics of the datasets? Which operating principles seem to be the most efficient and useful in general?

3.1. The top-performing oversamplers

In order to remove the dataset dimension of the results, the mean scores were computed over all datasets for each performance measure, oversampler, and classifier. We emphasize that the aggregation is carried out after model selection. The top-performing 10% of oversamplers regarding the various combinations of performance measures and classifiers are rendered in Table 3 with the scores of SMOTE and *classification without oversampling* as baseline results. The first thing to observe is that the classification performance without oversampling is significantly worse than using oversampling, which validates the use of oversamplers to improve imbalanced classification. According to the expectations, the top performers depend on the classifier and the measure. As a contribution to the field, Table 3 can be used as a reference to select oversampling techniques to be included in certain imbalanced learning pipelines.

One can also observe that the top performing advanced oversampling techniques outperform ordinary SMOTE with the improvement of about 1 percent in terms of AUC and F1, 2 percents in terms of G, and 0.25 percent in the case of P20. The advantage of using advanced oversamplers is less remarkable than expected. Previous papers have reported significant improvements compared to SMOTE, which seems to decrease by averaging over the unprecedented number of 104 imbalanced datasets. For example, the recent paper [19] uses 21 datasets for evaluation and reports an improvement of 0.04 AUC by SMOTE-PSO over SMOTE using SVM, while averaging over 104 datasets with thorough cross-validation leads to the improvement of 0.01 AUC by the best performer DBSMOTE over SMOTE. On the other hand, the high performance of some oversamplers seems to be consistent: techniques like polynom-fit-SMOTE are among the top performers in many respects.

In order to get a more explicit overall ranking, we have aggregated the results by averaging performance measures over classifiers, assigning the ranks of the average scores to each oversampler and taking the average of ranks (averaging ranks was found to be more meaningful than averaging scores of highly different meanings F1 and AUC). The final ranking of the overall top performers is given in Table 4: based on the evaluation methodology outlined before, these 10 techniques are the best candidates to be used on arbitrary, unseen data. In the next subsection, we provide some insight into the operation of the top three performers.

3.2. Analysis of the top performers

Polynom-fit-SMOTE refers to 4 fairly different oversampling strategies controlled by the *topology* parameter

of the technique. Checking the parameter settings providing the highest scores, we found that the various topologies are equally likely. The common behavior of using the 'bus', 'star', 'mesh' and 'polynom' topologies is that each of them generates instances along line segments between relatively far samples of the minority class, thus, the generated instances are more scattered in the manifold of the minority class, than using simple SMOTE.

In Proximity Weighted Synthesis (ProWSyn) the number of instances generated in the neighborhood of a minority sample is inversely proportional to its distance to majority instances. What makes ProWSyn unique among other sampling density based techniques is that it generates new instances by sampling the line segments between minority instances having similar distances to majority instances. This property makes ProWSyn similar to polynom-fit-SMOTE in the sense that samples are generated between relatively far minority instances, which seems to be an efficient oversampling approach even though the assumption made on the distribution of the data is stronger than that of SMOTE.

Finally, SMOTE-IPF executes ordinary SMOTE to generate new minority samples, then, uses a supervised classifier with cross-validation to check the consistency of new samples and remove the ones with low probabilities of belonging to the minority class. The default supervised classifier used in SMOTE-IPF is DT. We also mention that the 4th top performer technique by Lee et al. (Lee) implements a very similar approach using kNN for noise filtering. As a consequence, supervised classification based noise-filtering as a post-processing step seems to be another powerful operating principle for oversamplers.

The common property of the best performing oversamplers is that they are fairly simple, yet robust algorithms, if there are at least two different minority samples, these techniques are able to generate a balanced dataset without cloning. The reason why not math-intensive techniques are topping the rankings is that they are likely to fail in edge cases, for example, when the number of minority samples is extremely low (5-10) and the number of attributes is higher than the number of instances: clustering becomes unreliable, all minority samples can be identified as noise, autoencoders do not converge, and linear algebraic problems might become ill-posed.

One can expect that the performance of oversampling techniques depends on the characteristics of the datasets. In the next subsection, we examine the results by the most characteristic properties of datasets.

3.3. The top-performing oversamplers on various types of datasets

Databases are characterized by three main attributes in this study: the imbalance rate (IR), the number of minority samplers, and the number of attributes. We have introduced some thresholds and consider databases to either have high (IR > 9) or low imbalance rate, high ($N_+ > 30$) or low number of minority samplers and high ($d > 10$)

Table 3: The top performer oversamplers ranked by the average scores on all datasets: the columns report classification techniques and the four set of rows are related to the four performance measures; in each cell, the oversampling techniques providing the top results are reported, ranked in descending order

classifier	SVM		DT		kNN		MLP	
rank	sampler	AUC	sampler	AUC	sampler	AUC	sampler	AUC
1	DBSMOTE [12]	.9089	LVQ-SMOTE [78]	.8885	AHC [25]	.9153	CCR [57]	.9182
2	CURE-SMOTE [74]	.9068	ProWSyn [6]	.8861	ProWSyn [6]	.9139	SMOTE-IPF [89]	.9174
3	G-SMOTE [91]	.9062	Borderline-SMOTE2 [47]	.8860	Assembled-SMOTE [111]	.9095	Gaussian-SMOTE [61]	.9171
4	CE-SMOTE [23]	.9056	polynom-fit-SMOTE [43]	.8841	SMOTE-IPF [89]	.9092	DBSMOTE [12]	.9169
5	MDO [1]	.9042	SMOBD [17]	.8835	CE-SMOTE [23]	.9091	Assembled-SMOTE [111]	.9168
6	CCR [57]	.9037	Lee [62]	.8834	Gaussian-SMOTE [61]	.9090	polynom-fit-SMOTE [43]	.9166
7	polynom-fit-SMOTE [43]	.9037	Assembled-SMOTE [111]	.8834	Lee [62]	.9088	CE-SMOTE [23]	.9166
8	SMOTE-ENN [8]	.9029	SMOTE-IPF [89]	.8828	Borderline-SMOTE1 [47]	.9088	G-SMOTE [91]	.9166
baseline	SMOTE	.8999	SMOTE	.8809	SMOTE	.9082	SMOTE	.9156
baseline	no sampling	.8568	no sampling	.8124	no sampling	.8488	no sampling	.8248
rank	sampler	G	sampler	G	sampler	G	sampler	G
1	polynom-fit-SMOTE [43]	.8672	DE-oversampling [22]	.8658	polynom-fit-SMOTE [43]	.8820	Selected-SMOTE [56]	.8781
2	DBSMOTE [12]	.8661	SMOTE-ENN [8]	.8611	SMOTE-ENN [8]	.8785	SMOBD [17]	.8781
3	SMOTE-ENN [8]	.8649	Lee [62]	.8598	ProWSyn [6]	.8780	polynom-fit-SMOTE [43]	.8780
4	G-SMOTE [91]	.8629	Borderline-SMOTE2 [47]	.8598	SMOTE-IPF [89]	.8766	SMOTE-IPF [89]	.8777
5	CURE-SMOTE [74]	.8625	SMOTE-IPF [89]	.8590	SVM-balance [37]	.8756	Lee [62]	.8774
6	SMOTE-IPF [89]	.8616	SMOBD [17]	.8584	Lee [62]	.8748	ADOMS [96]	.8769
7	ProWSyn [6]	.8614	LVQ-SMOTE [78]	.8584	Assembled-SMOTE [111]	.8743	ProWSyn [6]	.8768
8	Lee [62]	.8612	Assembled-SMOTE [111]	.8582	SMOTE-TomekLinks [8]	.8736	NDO-sampling [108]	.8767
baseline	SMOTE	.8595	SMOTE	.8563	SMOTE	.8733	SMOTE	.8748
baseline	no sampling	.5284	no sampling	.7237	no sampling	.7007	no sampling	.6005
rank	sampler	F1	sampler	F1	sampler	F1	sampler	F1
1	DBSMOTE [12]	.6774	ProWSyn [6]	.7006	polynom-fit-SMOTE [43]	.7276	Supervised-SMOTE [51]	.6784
2	polynom-fit-SMOTE [43]	.6763	Lee [62]	.6994	Supervised-SMOTE [51]	.7254	polynom-fit-SMOTE [43]	.6774
3	Supervised-SMOTE [51]	.6723	SMOBD [17]	.6994	CCR [57]	.7239	DBSMOTE [12]	.6749
4	CURE-SMOTE [74]	.6696	polynom-fit-SMOTE [43]	.6993	ProWSyn [6]	.7208	Assembled-SMOTE [111]	.6745
5	SMOTE-Cosine [56]	.6678	Assembled-SMOTE [111]	.6969	LLE-SMOTE [98]	.7208	SMOTE-IPF [89]	.6744
6	CCR [57]	.6677	Supervised-SMOTE [51]	.6962	Gaussian-SMOTE [61]	.7206	NEATER [4]	.6742
7	LLE-SMOTE [98]	.6677	SMOTE-IPF [89]	.6961	CURE-SMOTE [74]	.7201	ProWSyn [6]	.6739
8	G-SMOTE [91]	.6677	NRSBoundary-SMOTE [38]	.6957	CE-SMOTE [23]	.7200	SMOBD [17]	.6737
baseline	SMOTE	.6636	SMOTE	.6932	SMOTE	.7114	SMOTE	.6706
baseline	no sampling	.4692	no sampling	.6092	no sampling	.6351	no sampling	.4308
rank	sampler	P20	sampler	P20	sampler	P20	sampler	P20
1	polynom-fit-SMOTE [43]	.9918	LVQ-SMOTE [78]	.9908	DE-oversampling [22]	.9939	Gaussian-SMOTE [61]	.9956
2	DBSMOTE [12]	.9914	DE-oversampling [22]	.9904	polynom-fit-SMOTE [43]	.9927	DE-oversampling [22]	.9953
3	LVQ-SMOTE [78]	.9913	polynom-fit-SMOTE [43]	.9902	ProWSyn [6]	.9926	polynom-fit-SMOTE [43]	.9953
4	Gaussian-SMOTE [61]	.9912	E-SMOTE [28]	.9893	E-SMOTE [28]	.9925	DEAGO [9]	.9951
5	LLE-SMOTE [98]	.9911	Gaussian-SMOTE [61]	.9890	SVM-balance [37]	.9923	Selected-SMOTE [56]	.9951
6	SMOTE-ENN [8]	.9911	CCR [57]	.9888	Gaussian-SMOTE [61]	.9920	cluster-SMOTE [24]	.9951
7	MOT2LD [101]	.9910	SSO [88]	.9884	NDO-sampling [108]	.9919	SSO [88]	.9951
8	cluster-SMOTE [24]	.9910	LLE-SMOTE [98]	.9882	SMOTE-ENN [8]	.9919	LVQ-SMOTE [78]	.9950
baseline	SMOTE	.9896	SMOTE	.9862	SMOTE	.9913	SMOTE	.9947
baseline	no sampling	.3242	no sampling	.3033	no sampling	.3285	no sampling	.2864

Table 4: The top performer oversamplers ranked by the combination of all scores. Besides the combined ranking, the aggregated values of the measures and the corresponding ranks are also reported

rank	sampler	average score	AUC	AUC rank	G	G rank	F1	F1 rank	P20	P20 rank
1	polynom-fit-SMOTE [43]	2.50	0.9025	6	0.8708	1	0.6952	1	0.9925	2
2	ProWSyn [6]	4.50	0.9044	1	0.8684	4	0.6903	3	0.9911	10
3	SMOTE-IPF [89]	7.50	0.9026	5	0.8687	3	0.6879	9	0.9909	13
4	Lee [62]	8.00	0.9023	7	0.8683	5	0.6881	8	0.9910	12
5	SMOBD [17]	9.25	0.9022	8	0.8677	6	0.6889	4	0.9906	19
6	G-SMOTE [91]	13.50	0.9019	10	0.8651	18	0.6866	12	0.9908	14
7	CCR [57]	14.25	0.9021	9	0.8620	30	0.6879	10	0.9913	8
8	LVQ-SMOTE [78]	14.75	0.9028	3	0.8623	29	0.6836	24	0.9922	3
9	Assembled-SMOTE [111]	15.50	0.9027	4	0.8669	7	0.6886	5	0.9827	46
10	SMOTE-TomekLinks [8]	15.75	0.9010	14	0.8662	9	0.6847	20	0.9906	20

or low number of attributes. The threshold on IR is supported by [3] which defines extreme imbalance as IR higher than 9, the other thresholds are empirical. The results are aggregated (like in Table 4) and only the final rankings are rendered in Table 5, where all techniques are listed which got into the top 10 for any type of dataset. The overall ranking is identical to that in Table 4.

The first thing to observe is that 66 techniques did not get in the top 10 performers on any type of dataset: 19 oversampling techniques provide the top 10 performers in each of the categories. One can also observe that the top 3 performers on all datasets (Table 4) perform among the top 6 for all types of datasets, suggesting that the overall ranking discussed in the previous subsection is robust and reliable, regardless of the characteristics of the datasets. The rankings on datasets with high IR and low N_+ are highly similar to the overall ranking, these types of datasets give the majority of the datasets involved in the study. In the next paragraphs, we examine the outliers giving superior results on some special types of datasets.

When the imbalance ratio is relatively low, some techniques like NEATER, Supervised-SMOTE, and cluster-SMOTE give superior results. Supervised-SMOTE is highly similar to SMOTE-IPF except SMOTE-IPF uses a single decision tree and removes noise after sample generation, while Supervised-SMOTE uses random forest by default and removes noise before sample generation. Supervised-SMOTE works properly only if the training of the random forest (which takes place before oversampling) is not skewed enormously by the imbalance: relatively low imbalance rate is required, just like confirmed by the test results. In many senses, NEATER is also similar to SMOTE-IPF, namely, it generates samples by ordinary SMOTE and ADASYN, and uses game theoretical approaches to remove inconsistencies. The results suggest that the game theory based filtering works well when the imbalance is moderate, however, the overall rank of 41 shows that it cannot cope with highly imbalanced data. Cluster-SMOTE applies k-means clustering to the minority samples and does a SMOTE-like sampling within the clusters. The good performance on low IR datasets can be attributed to two factors. On the one hand, low IR datasets in this study usually contain at least 50 minority samples (see Table 2),

thus, meaningful clusters are likely to be identified; on the other hand, the low IR implies that a relatively small number of samples need to be generated to balance the dataset, thus, the clusters do not get overly exaggerated and the minority class does not fall apart to the support regions of the clusters (see Figure 3 for an illustration of the deteriorating effect of overly exaggerated clusters). In the case of high IR, clustering techniques are likely to underperform other techniques as generating too many samples within small clusters deteriorates the generalization of the minority class by overemphasizing the clusters as distinct positive regions.

Finally, CBSO shows an increased performance when the number of attributes gets high. CBSO carries out agglomerative clustering and does a SMOTE-like sampling within the clusters, but generates samples only in the neighborhoods of minority samples having majority neighbors. The increased performance can be attributed to two main factors. On the one hand, in high dimensional spaces more minority instances have majority neighbors, thus, CBSO is less likely to overemphasize small regions near the class boundary. On the other hand, the higher the dimension of the data is, the more similar the distances between points become [58], thus, agglomerative clustering is likely to identify a low number of cluster and generating samples within the cluster means leads to samples generated between relatively far instances, similarly to polynom-fit-SMOTE and ProWSyn.

3.4. The performance of oversamplers by operating principles

In this subsection, we try to gain some insight into the performance related to the various operating principles of oversampling techniques. Although an analysis like this has many pitfalls as the effects of operating principles can hardly be separated from each other, we make an attempt to carry out a meaningful comparison. We decided to use the median scores within the categories as a measure to compare operating principles: they are not affected by extremely high and low performances and the median score of many techniques implementing some principle can be expected to characterize the principle to some extent by

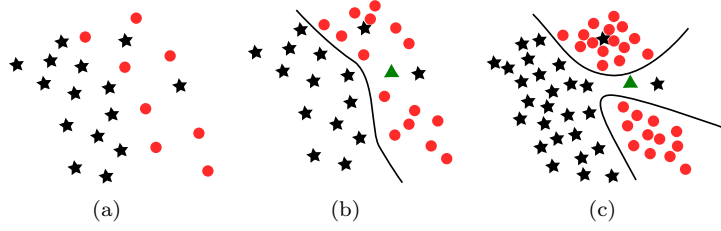


Figure 3: An imbalanced dataset with the relatively low imbalance rate $IR=2$ and two noisy majority samples (a). A clustering based oversampler seeking for 2 clusters can identify the top 4 and bottom 4 minority samples as clusters. Balancing the dataset by generating samples within the clusters keeps the common sense classification boundary depicted and classifies the test point (green triangle) as a minority sample (b). If the imbalance rate is extreme, balancing the dataset overly exaggerates the two clusters, makes classifiers learn the clusters and classify the test point as a majority sample (c).

Table 5: Comparison of the performance of oversampling techniques on various types of datasets

	sampler	overall rank	$IR > 9$	$IR \leq 9$	$N_+ > 30$	$N_+ \leq 30$	$d > 10$	$d \leq 10$
1	polynom-fit-SMOTE [43]	1	1	1	3	1	1	1
2	ProWSyn [6]	2	2	2	1	3	2	2
3	SMOTE-IPF [89]	3	3	4	2	6	5	3
4	Lee [62]	4	4	4	4	4	4	4
5	SMOBD [17]	5	5	8	5	7	6	8
6	G-SMOTE [91]	6	8	9	13	8	7	10
7	CCR [57]	7	6	21	14	9	20	7
8	LVQ-SMOTE [78]	8	7	16	21	2	19	5
9	Assembled-SMOTE [111]	9	9	10	16	5	9	6
10	SMOTE-TomekLinks [8]	10	12	14	10	15	12	13
11	SMOTE [20]	11	12	26	8	18	12	17
12	Random-SMOTE [30]	12	14	26	8	19	15	16
13	CE-SMOTE [23]	14	10	36	26	10	8	27
14	SMOTE-Cosine [56]	15	18	14	6	21	28	9
15	Selected-SMOTE [56]	19	21	12	8	23	10	20
16	Supervised-SMOTE [51]	22	24	6	12	28	18	21
17	CBSO [5]	26	31	28	22	33	3	34
18	cluster-SMOTE [24]	26	29	7	20	31	29	24
19	NEATER [4]	41	42	3	48	40	46	20

smoothing other effects. The results are rendered in Table 6.

One can observe that the rankings by the AUC, G and F1 scores are highly similar, while P20 is slightly different from them: borderline techniques rank only as 9th but the ones using classifiers top the ranking. Disregarding these anomalies with P20, there seems to be an agreement on the performance of various operating principles.

The topping of *borderline* and *density based* techniques means that it does worth sampling near the decision boundary, in general. Sampling near the class boundary naturally improves instance-based classification (like kNN and SVM), aids the proper partitioning of the space when DTs are used, and just like all oversamplers, it helps MLP as more minority samples are represented in the back-propagation gradient. From the overall top performers ProWSyn, SMOBD, and Assembled-SMOTE implement density based oversampling.

Ordinary sampling performs better than *component-wise sampling*, and the principle of *sampling by cloning* is among the worst performers. As discussed before, component-wise sampling makes stronger, while cloning makes milder assumptions on the distribution of the data than ordi-

nary sampling. The consequence to draw is that ordinary sampling makes the right compromise between introducing variance and staying close to the original distribution.

One interesting result is that the category *application* is close to the top performers, indicating that oversamplers developed for specific applications are worth to be considered for other applications.

The moderate performance of *clustering based techniques* is explained by the issues already discussed: sample generation within clusters might overemphasize clusters when the imbalance ratio is high and deteriorate the subsequent classification results. Similarly to clustering, the moderate performance of *density estimation* and *dimension reduction* based techniques indicates that in some cases they work well, but the performance drops in oversampler-specific edge cases. Consequently, the automated application of techniques like these necessitates the careful monitoring of results.

Finally, the techniques using *memetic algorithms* give the worst performance, and they are also responsible for low scores of the techniques *using classifiers*. We emphasize that these techniques have the highest number of parameters, which would necessitate a more careful model se-

lection than in other cases. The proper, manual tuning of parameters might yield better results, however, when machine learning needs to be done at scale to process many datasets in an automated manner, techniques requiring manual tuning might be unfavorable.

At this point, we revisit the anomalies in the results of the P20 measure: the extremely high performance of *classifier based* techniques and the low performance of *borderline* techniques needs explanation. Unlike the other measures, P20 does not take into consideration the classification performance on majority samples. One can readily see, that the oversamplers using classifiers to remove samples with low positive class probabilities (like SMOTE-IPF and Lee) optimize exactly the P20 measure: they keep minority samples belonging to the minority class with high probabilities. Due to the relatively large number of techniques like these having the same high P20 scores, the performance of *memetic optimization based methods* (also using classifiers) is hidden by the median score of the category, and consequently, the classifier based techniques top the P20 ranking. On the other hand, the low P20 performance of borderline techniques can be explained by the fact that generating too many samples near the decision boundary might turn truly majority samples into minority ones with high probability, hence, reducing the P20 scores in the category. Finally, we highlight that the P20 score might be a good choice to optimize the parameters of one specific oversampler, but care must be taken when conclusions are drawn from P20 scores as the results depend on the operating principles.

3.5. Runtimes

Runtime is another important feature of oversampling techniques as some applications might require rapid re-training, thus, rapid oversampling. Given the large number of algorithms and parameters affecting time complexities, deriving and reporting time complexities in big-O notation is beyond the scope of this study. However, we expect that the average runtimes of oversamplers (rendered in Table 7) can still provide meaningful insight into their time efficiency, nevertheless, we emphasize that runtimes always depend on the implementations.

According to the expectations, memetic oversamplers have the longest runtimes as they carry out an expensive optimization driven by cross-validation. All the top performers discussed in the previous subsections have moderate runtimes. From the overall top 10 performers (Table 4), the fastest technique is SMOBD, being 50% quicker than the overall top performer polynom-fit-SMOTE, however, SMOBD is still 8 times slower in average than the fastest technique SPY. The consequence we can draw is that the best performing techniques are definitely not the computationally most intensive ones, however, if extremely fast techniques are needed, oversamplers up to 5-10 times faster than the top performers are available for the price of a decrease in quality.

4. Discussion and conclusions

The primary goal of this study is to carry out a detailed comparison of 85 oversampling techniques involving 104 imbalanced datasets. Further goals are gaining insight into the performance of oversamplers on various types of datasets, and the performance of various operating principles. The key findings and contributions of the paper can be summarized as follows:

1. We have determined the oversamplers providing the best performance respecting 4 commonly used classification techniques and 4 performance measures of imbalanced learning. The results summarized in Table 3 can be used as a reference to select the most suitable oversamplers for existing machine learning pipelines.
2. Comparing the results of oversamplers to the baseline *no oversampling* in Table 3 justifies and confirms that oversampling is a reasonable technique to improve the classification performance on imbalanced datasets.
3. We have identified the oversampling techniques giving the best performance irrespectively of classifier types and performance measures. The results in Table 4 suggest that polynom-fit-SMOTE, ProWSyn and SMOTE-IPF generate high-quality samples, these techniques are recommended to be applied to data with unknown or continuously changing characteristics.
4. Comparing the performance of oversamplers on various types of datasets (see Table 5) shows that some relatively simple techniques provide highly reliable performance regardless of the characteristic properties of the datasets. The performance of math-intensive techniques is likely to fail in edge cases, for example, at extreme imbalance rates.
5. Comparing the performances related to various operating principles (see Table 6) suggests that the most successful principles are ordinary sampling (sampling line segments between neighboring instances), and borderline/density based sampling (generating more instances near the class boundaries). We note that the common sampling strategy of the overall top performers (sampling along line segments between relatively far minority samples) does not fall any of the introduced categories. These findings suggest a promising research direction: efficient general purpose oversampling techniques should sample the line segments connecting relatively far instances and generate more instances near the borderline.
6. Although the runtimes shared in Table 7 are related to our implementation, they still provide an insight into the time-efficiency of various oversamplers and enable making compromises between performance and speed for time-efficient applications.

Table 6: Comparison of operating principles

rank	attribute AUC	attribute G	attribute F1	attribute P20
1	borderline .8992	ordinary sampling .8626	ordinary sampling .6799	uses classifier .9903
2	ordinary sampling .8985	borderline .8617	application .6765	density based .9901
3	density based .8980	density based .8608	uses clustering .6762	ordinary sampling .9899
4	application .8954	application .8578	density based .6759	componentwise sampling .9896
5	density estimation .8947	componentwise sampling .8443	borderline .6757	application .9895
6	componentwise sampling .8944	dimension reduction .8442	uses classifier .6701	density estimation .9895
7	dimension reduction .8926	uses clustering .8415	density estimation .6694	dimension reduction .9874
8	uses clustering .8913	uses classifier .8322	componentwise sampling .6671	uses clustering .9845
9	uses classifier .8898	density estimation .8258	dimension reduction .6562	borderline .9652
10	sampling by cloning .8793	memetic .8206	sampling by cloning .6496	sampling by cloning .6671
11	memetic .8605	sampling by cloning .8101	memetic .6435	memetic .6286

Table 7: Average runtime of oversampling techniques in seconds

oversampler	time	oversampler	time	oversampler	time
SPY	0.11	ANS	0.64	SMOTE_Cosine	2.01
OUPS	0.17	MSMOTE	0.73	kmeans_SMOTE	2.44
SMOTE_D	0.21	Safe_Level_SMOTE	0.79	MWMOTE	2.45
NT_SMOTE	0.21	SMOBD	0.80	SMOTE_FRST_2T	2.59
Gazzah	0.21	CBSO	0.82	SMOTE_ENN	2.76
ROSE	0.26	Assembled_SMOTE	0.82	A_SUWO	2.81
Borderline_SMOTE1	0.28	SDSMOTE	0.88	RWO_sampling	2.92
Borderline_SMOTE2	0.30	Edge_Det_SMOTE	0.94	SMOTE_IPF	3.82
ISMOTE	0.30	SMOTE_TomekLinks	0.99	ADOMS	3.89
SMMO	0.32	ProWSyn	1.00	Lee	4.16
SMOTE_OUT	0.38	Stefanowski	1.04	cluster_SMOTE	4.19
SN_SMOTE	0.44	AND_SMOTE	1.14	SOMO	4.31
NDO_sampling	0.45	DBSMOTE	1.18	DE_oversampling	4.68
SMOTE	0.46	polynom_fit_SMOTE	1.18	CCR	4.72
Selected_SMOTE	0.47	ASMOBD	1.19	SMOTE_RSB	5.12
distance_SMOTE	0.47	MDO	1.19	V_SYNTH	5.23
Gaussian_SMOTE	0.49	SOLCJ	1.24	NRSBoundary_SMOTE	5.27
MCT	0.52	LN_SMOTE	1.27	AHC	5.28
Random_SMOTE	0.58	VIS_RST	1.35	LVQ_SMOTE	7.00
ADASYN	0.59	TRIM_SMOTE	1.37	ISOMAP_Hybrid	7.01
SL_graph_SMOTE	0.59	NRAS	1.56	CE_SMOTE	7.45
CURE_SMOTE	0.59	LLE_SMOTE	1.63	MSYN	13.96
				PDFOS	15.14
				KernelADASYN	17.87
				G_SMOTE	19.24
				SVM_balance	22.97
				E_SMOTE	26.13
				SUNDO	26.21
				GASMOTE	31.39
				DEAGO	38.52
				SMOTE_PSO	45.12
				NEATER	75.59
				IPADE_ID	90.01
				DSMOTE	146.73
				MOT2LD	149.42
				Supervised_SMOTE	195.75
				SSO	215.28
				SMOTE_PSOBAT	331.32
				DSRBF	383.36
				ADG	493.64
				AMSCO	659.02

7. The improvements achieved by advanced oversamplers compared to SMOTE are less significant than the improvements achieved by any reliable oversampler compared to neglecting oversampling. The performances of advanced oversampling techniques seem to level when averaging over 104 datasets. We can draw the consequence that no oversampler can provide really outstanding performance on each dataset, thus, oversamplers need to be designed to operate on datasets with certain intrinsic characteristics and tests need to be provided to check if a dataset is suitable to be oversampled by a particular technique. For example, for cluster-based oversamplers it would be reasonable to select a lower limit on the information theoretical gains of clustering which ensures that clusters are really present in the data and cluster-based oversampling is applicable.

An interesting observation is that none of the recently published techniques have been compared to any of the top performers in the corresponding papers. The comparison of new techniques to a conventional subset of historical ones draws back the development in the field as the

conventional techniques get more and more acknowledged while well-performing but less popular recent algorithms get forgotten.

The comparison of 85 algorithms is a challenging task, the results suffer from several limitations: (a) reproducing algorithms from research papers is prone to errors; (b) finding a well-performing set of parameters for each database is challenging – improved model selection might slightly change the results, especially for oversamplers with many hyperparameters; (c) although we have involved 104 datasets in the experiments, application specific datasets might have highly different characteristics, thus, proper oversampler and model selection is still recommended. Nevertheless, we believe that all the conclusions we drew are valid in the scope of the paper and the results might help both theorists and practitioners to develop the field.

For reproducibility, the foldings of datasets, the implementation of oversamplers and all the evaluation scripts have been shared at http://github.com/gykovacs/smote_variants. Additionally, all numerical results have been shared in the same repository to enable further analysis by the community.

Future steps include the development of general purpose oversampling techniques based on the consequences drawn in this study, and the evaluation of classifier ensembles where classifiers are trained on oversampled datasets obtained by different oversampling techniques.

References

- [1] Abdi, L., Hashemi, S., 2016. To combat multi-class imbalanced problems by means of over-sampling techniques. *IEEE Trans Knowl and Data Eng* 28, 238–251.
- [2] Al-Khaldy, M., Kambhampati, C., 2018. Resampling imbalanced class and the effectiveness of feature selection methods for heart failure dataset. *International Robotics and Automation Journal* 4, 1–10.
- [3] Alcalá-Fdez, J., Fernández, A., Luengo, J., Derrac, J., García, S., Sánchez, L., Herrera, F., 2011. KEEL Data-Mining Software Tool: Data set repository, integration of algorithms and experimental analysis framework. *Journal of Multiple-Valued Logic and Soft Computing* 17, 255–287.
- [4] Almogahed, B.A., Kakadiaris, I.A., 2014. NEATER: Filtering of over-sampled data using non-cooperative game theory, in: 22nd International Conference on Pattern Recognition, pp. 1371–1376.
- [5] Barua, S., Islam, M.M., Murase, K., 2011. A novel synthetic minority oversampling technique for imbalanced data set learning, in: *Neural Information Processing*, Springer Berlin Heidelberg. pp. 735–744.
- [6] Barua, S., Islam, M.M., Murase, K., 2013. ProWSyn: Proximity weighted synthetic oversampling technique for imbalanced data set learning, in: *Advances in Knowledge Discovery and Data Mining*, Springer Berlin Heidelberg. pp. 317–328.
- [7] Barua, S., Islam, M.M., Yao, X., Murase, K., 2014. MWMOTE—majority weighted minority oversampling technique for imbalanced data set learning. *IEEE Trans Knowl and Data Eng* 26, 405–425.
- [8] Batista, G.E.A.P.A., Prati, R.C., Monard, M.C., 2004. A study of the behavior of several methods for balancing machine learning training data. *SIGKDD Explor. Newsl.* 6, 20–29.
- [9] Bellinger, C., Japkowicz, N., Drummond, C., 2015. Synthetic oversampling for advanced radioactive threat detection, in: *IEEE 14th International Conference on Machine Learning and Applications*, pp. 948–953.
- [10] Borowska, K., Stepaniuk, J., 2016. Imbalanced data classification: A novel re-sampling approach combining versatile improved SMOTE and rough sets, in: *Computer Information Systems and Industrial Management*, Springer Int. Publishing. pp. 31–42.
- [11] Bunkhumpornpat, C., Sinapiromsaran, K., Lursinsap, C., 2009. Safe-Level-SMOTE: Safe-level-synthetic minority oversampling technique for handling the class imbalanced problem, in: *Proc. of the 13th Pacific-Asia Conference on Advances in Knowledge Discovery and Data Mining*, pp. 475–482.
- [12] Bunkhumpornpat, C., Sinapiromsaran, K., Lursinsap, C., 2012. DBSMOTE: Density-based synthetic minority oversampling technique. *Applied Intelligence* 36, 664–684.
- [13] Bunkhumpornpat, C., Subpaiboonkit, S., 2013. Safe level graph for synthetic minority over-sampling techniques, in: *13th International Symposium on Communications and Information Technologies*, pp. 570–575.
- [14] de la Calleja, J., Fuentes, O., 2007. A distance-based oversampling method for learning from imbalanced data sets, in: *Proceedings of the Twentieth International Florida Artificial Intelligence*, pp. 634–635.
- [15] de la Calleja, J., Fuentes, O., González, J., 2008. Selecting minority examples from misclassified data for over-sampling, in: *Proc. of the 21st Int. Florida Artificial Intelligence Research Society Conference*, pp. 276–281.
- [16] Cao, P., Liu, X., Zhang, J., Zhao, D., Huang, M., Zaiane, O., 2017. 2,1 norm regularized multi-kernel based joint nonlinear feature selection and over-sampling for imbalanced data classification. *Neurocomputing* 234, 38 – 57.
- [17] Cao, Q., Wang, S., 2011. Applying over-sampling technique based on data density and cost-sensitive SVM to imbalanced learning, in: *International Conference on Information Management, Innovation Management and Industrial Engineering*, pp. 543–548.
- [18] Cateni, S., Colla, V., Vannucci, M., 2011. Novel resampling method for the classification of imbalanced datasets for industrial and other real-world problems, in: *2011 11th International Conference on Intelligent Systems Design and Applications*, pp. 402–407.
- [19] Cervantes, J., García-Lamont, F., Rodríguez, L., López, A., Castilla, J.R., Trueba, A., 2017. PSO-based method for SVM classification on skewed data sets. *Neurocomputing* 228, 187 – 197.
- [20] Chawla, N.V., Bowyer, K.W., Hall, L.O., Kegelmeyer, W.P., 2002. SMOTE: synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research* 16, 321–357.
- [21] Chawla, N.W., 2010. Data mining for imbalanced datasets: an overview. Springer. pp. 875–886.
- [22] Chen, L., Cai, Z., Chen, L., Gu, Q., 2010a. A novel differential evolution-clustering hybrid resampling algorithm on imbalanced datasets, in: *3rd International Conference on Knowledge Discovery and Data Mining*, pp. 81–85.
- [23] Chen, S., Guo, G., Chen, L., 2010b. A new over-sampling method based on cluster ensembles, in: *2010 IEEE 24th International Conference on Advanced Information Networking and Applications Workshops*, pp. 599–604.
- [24] Cieslak, D.A., Chawla, N.V., Striegel, A., 2006. Combating imbalance in network intrusion datasets, in: *2006 IEEE International Conference on Granular Computing*, pp. 732–737.
- [25] Cohen, G., Hilario, M., Sax, H., Hugonnet, S., Geissbühler, A., 2006. Learning from imbalanced data in surveillance of nosocomial infection. *Artif Intell Med* 37, 7 – 18.
- [26] Dang, X.T., Tran, D.H., Hirose, O., Satou, K., 2015. SPY: A novel resampling method for improving classification performance in imbalanced data, in: *2015 Seventh International Conference on Knowledge and Systems Engineering (KSE)*, pp. 280–285.
- [27] Das, B., Krishnan, N.C., Cook, D.J., 2015. RACOG and wRACOG: Two probabilistic oversampling techniques. *IEEE Trans on Knowl and Data Eng* 27, 222–234.
- [28] Deepa, T., Punithavalli, M., 2011. An E-SMOTE technique for feature selection in high-dimensional imbalanced dataset, in: *2011 3rd International Conference on Electronics Computer Technology*, pp. 322–324.
- [29] Dheeru, D., Karra Taniskidou, E., 2017. UCI machine learning repository. URL: <http://archive.ics.uci.edu/ml>.
- [30] Dong, Y., Wang, X., 2011. A new over-sampling approach: Random-SMOTE for learning from imbalanced data sets, in: *Knowledge Science, Engineering and Management*, Springer Berlin Heidelberg. pp. 343–352.
- [31] Douzas, G., Bacao, F., 2017. Self-organizing map oversampling (SOMO) for imbalanced data set learning. *Expert Syst Appl* 82, 40 – 52.
- [32] Douzas, G., Bacao, F., 2018. Effective data generation for imbalanced learning using conditional generative adversarial networks. *Expert Syst. Appl.* 91, 464–471.
- [33] Douzas, G., Bacao, F., Last, F., 2018. Improving imbalanced learning through a heuristic oversampling method based on k-means and SMOTE. *Inf Sci* 465, 1 – 20.
- [34] van Dyk, D.A., Meng, X.L., 2001. The art of data augmentation. *Journal of Computational and Graphical Statistics* 10, 1–50.
- [35] Ertekin, Ş., 2013. Adaptive oversampling for imbalanced data classification, in: *Information Sciences and Systems 2013*, Springer International Publishing. pp. 261–269.
- [36] Fan, X., Tang, K., Weise, T., 2011. Margin-based oversampling method for learning from imbalanced datasets, in: *Advances in Knowledge Discovery and Data Mining*, Springer

Berlin Heidelberg. pp. 309–320.

- [37] Farquard, M.A.H., Bose, I., 2012. Preprocessing unbalanced data using support vector machine. *Decis. Support Syst.* 53, 226–233.
- [38] Feng, H., Hang, L., 2013. A novel boundary oversampling algorithm based on neighborhood rough set model: NRSBoundary-SMOTE. *Mathematical Problems in Engineering*, 694809.
- [39] Fernandez, A., Garcia, S., Herrera, F., Chawla, N.V., 2018. SMOTE for learning from imbalanced data: Progress and challenges, marking the 15-year anniversary. *Journal of Artificial Intelligence Research* 61, 863–905.
- [40] Fernández-Navarro, F., Hervás-Martínez, C., Gutiérrez, P.A., 2011. A dynamic over-sampling procedure based on sensitivity for multi-class problems. *Pattern Recognition* 44, 1821 – 1833.
- [41] Gao, M., Hong, X., Chen, S., Harris, C.J., Khalaf, E., 2014. PDFOS: PDF estimation based over-sampling for imbalanced two-class problems. *Neurocomputing* 138, 248 – 259.
- [42] García, V., Sánchez, J.S., Martín-Félez, R., Mollineda, R.A., 2012. Surrounding neighborhood-based SMOTE for learning from imbalanced data sets. *Progress in Artificial Intelligence* 1, 347–362.
- [43] Gazzah, S., Amara, N.E.B., 2008. New oversampling approaches based on polynomial fitting for imbalanced data sets in: 2008 The Eighth IAPR International Workshop on Document Analysis Systems, pp. 677–684.
- [44] Gazzah, S., Heckel, A., Amara, N.E.B., 2015. A hybrid sampling method for imbalanced data, in: IEEE 12th International Multi-Conference on Systems, Signals Devices, pp. 1–6.
- [45] Gosain, A., Sardana, S., 2017. Handling class imbalance problem using oversampling techniques: A review, in: 2017 International Conference on Advances in Computing, Communications and Informatics (ICACCI), pp. 79–85.
- [46] Gu, Q., Cai, Z., Zhu, L., 2009. Classification of imbalanced data sets by using the hybrid re-sampling algorithm based on Isomap, in: Proceedings of the 4th International Symposium on Advances in Computation and Intelligence, Springer-Verlag. pp. 287–296.
- [47] Han, H., Wang, W.Y., Mao, B.H., 2005. Borderline-SMOTE: A new over-sampling method in imbalanced data sets learning, in: Advances in Intelligent Computing, Springer Berlin Heidelberg. pp. 878–887.
- [48] He, H., Bai, Y., Garcia, E.A., Li, S., 2008. ADASYN: adaptive synthetic sampling approach for imbalanced learning, in: Proc of IJCNN, pp. 1322–1328.
- [49] He, H., Gracia, E.A., 2009. Learning from imbalanced data. *IEEE Transactions on Knowledge Discovery* 21, 1263–1284.
- [50] Hoens, T.R., Chawla, N.V., 2013. Imbalanced Datasets: From Sampling to Classifiers. John Wiley & Sons, Ltd. chapter 3, pp. 43–59.
- [51] Hu, J., He, X., Yu, D.J., Yang, X.B., Yang, J.Y., Shen, H.B., 2014. A new supervised over-sampling algorithm with application to protein-nucleotide binding residue prediction. *PLOS ONE* 9, 1–10.
- [52] Hu, S., Liang, Y., Ma, L., He, Y., 2009. MSMOTE: Improving classification performance when training data is imbalanced, in: Proc. of the 2nd International Workshop on Computer Science and Engineering - Vol. 2, pp. 13–17.
- [53] Jiang, K., Lu, J., Xia, K., 2016. A novel algorithm for imbalanced data classification based on genetic algorithm improved SMOTE. *Arabian Journal for Science and Engineering* 41, 3255–3266.
- [54] Jiang, L., Qiu, C., Li, C., 2015. A novel minority cloning technique for cost-sensitive learning. *International Journal of Pattern Recognition and Artificial Intelligence* 29, 1551004.
- [55] Kang, Y., Won, S., 2010. Weight decision algorithm for oversampling technique on class-imbalanced learning, in: ICCAS 2010, pp. 182–186.
- [56] Koto, F., 2014. SMOTE-Out, SMOTE-Cosine, and Selected SMOTE: An enhancement strategy to handle imbalance in data level. 2014 Int. Conf. on Advanced Computer Science and Information System, 280–284.
- [57] Koziarski, M., Wozniak, M., 2017. Ccr: A combined cleaning and resampling algorithm for imbalanced data classification. *International Journal of Applied Mathematics and Computer Science* 27, 727–736.
- [58] Kriegel, H.P., Kröger, P., Zimek, A., 2009. Clustering high-dimensional data: A survey on subspace clustering, pattern-based clustering, and correlation clustering. *ACM Trans. Knowl. Discov. Data* 3, 1:1–1:58.
- [59] Kukar, M., Kononenko, I., 1998. Cost-sensitive learning with neural networks, in: Proceedings of the 13th European Conference on Artificial Intelligence (ECAI-98), John Wiley and Sons. pp. 445–449.
- [60] László, Z., Török, L., Kovács, G., 2018. Improving the performance of the k rare class nearest neighbor classifier by the ranking of point patterns, in: Proc. of Foundations of Information and Knowledge Systems, pp. 265–283.
- [61] Lee, H., Kim, J., Kim, S., 2017. Gaussian-based SMOTE algorithm for solving skewed class distributions. *Int. J. Fuzzy Logic and Intell Sys* 17, 229–234.
- [62] Lee, J., Kim, N., Lee, J.H., 2015. An over-sampling technique with rejection for imbalanced class learning, in: Proceedings of the 9th International Conference on Ubiquitous Information Management and Communication, ACM, New York, NY, USA. pp. 102:1–102:6.
- [63] Lemaître, G., Nogueira, F., Aridas, C.K., 2017. Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. *J Mach Learn Res* 18, 1–5.
- [64] Li, H., Zou, P., Wang, X., Xia, R., 2013. A new combination sampling method for imbalanced data, in: Proc. of 2013 Chinese Intelligent Automation Conference, Springer Berlin Heidelberg. pp. 547–554.
- [65] Li, J., Fong, S., Wong, R.K., Chu, V.W., 2018. Adaptive multi-objective swarm fusion for imbalanced data classification. *Inf Fusion* 39, 1 – 24.
- [66] Li, J., Fong, S., Zhuang, Y., 2015. Optimizing SMOTE by metaheuristics with neural network and decision tree, in: 2015 3rd International Symposium on Computational and Business Intelligence (ISCBI), pp. 26–32.
- [67] Li, K., Zhang, W., Lu, Q., Fang, X., 2014. An improved SMOTE imbalanced data classification method based on support degree, in: 2014 International Conference on Identification, Information and Knowledge in the Internet of Things, pp. 34–38.
- [68] Li, Y., Zhang, X., 2011. Improving k nearest neighbor with exemplar generalization for imbalanced classification, in: PAKDD 2011, pp. 1–12.
- [69] Liu, W., Chawla, S., 2011. Class confidence weighted kNN algorithms for imbalanced data sets, in: Proceedings of PAKDD 2011, pp. 354–356.
- [70] Lomax, S., Vadera, S., 2013. A survey of cost-sensitive decision tree induction algorithms. *ACM Computing Surveys* 45, 16:1–16:35.
- [71] Lopez, V., Fernandez, A., Garcia, S., Palade, V., Herrera, F., 2013. An insight into classification with imbalanced data: Empirical results and current trends on using data intrinsic characteristics. *Inf Sci* 250, 113 – 141.
- [72] Lopez, V., Fernandez, A., Herrera, F., 2014a. On the importance of the validation technique for classification with imbalanced datasets: Addressing covariate shift when data is skewed. *Inf Sci* 257, 1 – 13.
- [73] Lopez, V., Triguero, I., Carmona, C.J., Garcia, S., Herrera, F., 2014b. Addressing imbalanced classification with instance generation techniques: IPAD-ED. *Neurocomputing* 126, 15 – 28.
- [74] Ma, L., Fan, S., 2017. CURE-SMOTE algorithm and hybrid algorithm for feature selection and parameter optimization based on random forests. *BMC Bioinformatics* 18, 1–18.
- [75] Maciejewski, T., Stefanowski, J., 2011. Local neighbourhood extension of SMOTE for mining imbalanced data, in: 2011 IEEE Symposium on Computational Intelligence and Data Mining (CIDM), pp. 104–111.

- [76] Mahmoudi, S., Moradi, P., Akhlaghian, F., Moradi, R., 2014. Diversity and separable metrics in over-sampling technique for imbalanced data classification, in: 4th International Conference on Computer and Knowledge Engineering, pp. 152–158.
- [77] Menardi, G., Torelli, N., 2014. Training and assessing classification rules with imbalanced data. *Data Min Knowl Discov* 28, 92–122.
- [78] Nakamura, M., Kajiwar, Y., Otsuka, A., Kimura, H., 2013. LVQ-SMOTE – learning vector quantization based synthetic minority over-sampling technique for biomedical data. *Bio-Data Mining* 6, 1–10.
- [79] Nekooimehr, I., Lai-Yuen, S.K., 2016. Adaptive semi-supervised weighted oversampling (A-SUWO) for imbalanced datasets. *Expert Syst Appl* 46, 405 – 416.
- [80] Pourhabib, A., Mallick, B.K., Ding, Y., 2015. Absent data generating classifier for imbalanced class sizes. *J Mach Learn Res* 16, 2695–2724.
- [81] Puntumapon, K., Waiyamai, K., 2012. A pruning-based approach for searching precise and generalized region for synthetic minority over-sampling, in: *Advances in Knowledge Discovery and Data Mining*, Springer Berlin Heidelberg. pp. 371–382.
- [82] Qi, Z., Tian, Y., Shi, Y., Yu, X., 2013. Cost-sensitive support vector machine for semi-supervised learning. *Procedia Comput Sci* 18, 1684–1689.
- [83] Raeder, T., Forman, G., Chawla, N.V., 2012. Learning from imbalanced data: Evaluation matters, in: *Data Mining: Foundations and Intelligent Paradigms: Volume 1: Clustering, Association and Classification*, Springer Berlin Heidelberg. pp. 315–331.
- [84] Ramentol, E., Caballero, Y., Bello, R., Herrera, F., 2012. SMOTE-RSB*: a hybrid preprocessing approach based on oversampling and undersampling for high imbalanced datasets using SMOTE and rough sets theory. *Knowledge and Information Systems* 33, 245–265.
- [85] Ramentol, E., Gondres, I., Lajes, S., Bello, R., Caballero, Y., Cornelis, C., Herrera, F., 2016. Fuzzy-rough imbalanced learning for the diagnosis of high voltage circuit breaker maintenance: The SMOTE-FRST-2T algorithm. *Eng Appl Artif Intell* 48, 134 – 139.
- [86] Rivera, W.A., 2017. Noise reduction a priori synthetic oversampling for class imbalanced data sets. *Inf Sci* 408, 146 – 161.
- [87] Rivera, W.A., Xanthopoulos, P., 2016. A priori synthetic oversampling methods for increasing classification sensitivity in imbalanced data sets. *Expert Syst Appl* 66, 124 – 135.
- [88] Rong, T., Gong, H., Ng, W.W.Y., 2014. Stochastic sensitivity oversampling technique for imbalanced data, in: *Machine Learning and Cybernetics*, Springer Berlin Heidelberg. pp. 161–171.
- [89] Sáez, J.A., Luengo, J., Stefanowski, J., Herrera, F., 2015. SMOTE-IPF: Addressing the noisy and borderline examples problem in imbalanced classification by a re-sampling method with filtering. *Inf Sci* 291, 184 – 203.
- [90] Sanchez, I.A., Morales, E., Gonzalez, J., 2013. Synthetic oversampling of instances using clustering. *Int J of Artif Intell Tools* 22, 1350008.
- [91] Sandhan, T., Choi, J.Y., 2014. Handling imbalanced datasets by partially guided hybrid sampling for pattern recognition, in: 22nd International Conference on Pattern Recognition, pp. 1449–1453.
- [92] Shalev-Shwartz, S., Ben-David, S., 2014. *Understanding machine learning: from theory to algorithms*. Cambridge University Press.
- [93] Siriseriwan, W., Sinapiromsaran, K., 2017. Adaptive neighbor synthetic minority oversampling technique under 1NN outcast handling, in: *Songklanakarin Journal of Science and Technology*, pp. 565–576.
- [94] Stefanowski, J., Wilk, S., 2008. Selective pre-processing of imbalanced data for improving classification performance, in: *Proceedings of the 10th International Conference on Data Warehousing and Knowledge Discovery*, Springer-Verlag. pp. 283–292.
- [95] Tang, B., He, H., 2015. KernelADASYN: Kernel based adaptive synthetic data generation for imbalanced learning, in: *IEEE Congress on Evolutionary Computation*, pp. 664–671.
- [96] Tang, S., Chen, S., 2008. The generation mechanism of synthetic minority class examples, in: 2008 International Conference on Information Technology and Applications in Biomedicine, pp. 444–447.
- [97] Torres, F.R., Carrasco-Ochoa, J.A., Martínez-Trinidad, J.F., 2016. SMOTE-D a deterministic version of SMOTE, in: *MCPR2016: Pattern Recognition*, pp. 177–188.
- [98] Wang, J., Xu, M., Wang, H., Zhang, J., 2006. Classification of imbalanced data by using the SMOTE algorithm and locally linear embedding, in: 2006 8th international Conference on Signal Processing, pp. 1–9.
- [99] Wang, S., Li, Z., Chao, W., Cao, Q., 2012. Applying adaptive over-sampling technique based on data density and cost-sensitive SVM to imbalanced learning, in: *Int. Joint Conf. on Neural Networks*, pp. 1–8.
- [100] Wang, Y., Sun, S., Zhong, J., 2016. An ensemble anomaly detection with imbalanced data based on robot vision. *Int J Robot Autom* 31, 1–7.
- [101] Xie, Z., Jiang, L., Ye, T., Li, X., 2015. A synthetic minority oversampling method based on local densities in low-dimensional space for imbalanced learning, in: *Database Systems for Advanced Applications*, Springer International Publishing. pp. 3–18.
- [102] Xu, Y.H., Li, H., Le, L.P., Tian, X.Y., 2014. Neighborhood triangular synthetic minority over-sampling technique for imbalanced prediction on small samples of chinese tourism and hospitality firms, in: 7th Int. Joint Conf. on Computational Sciences and Optimization, pp. 534–538.
- [103] Young, W.A., Nykl, S.L., Weckman, G.R., Chelberg, D.M., 2015. Using Voronoi diagrams to improve classification performances when modeling imbalanced datasets. *Neural Comput. Appl.* 26, 1041–1054.
- [104] Yu, H., Ni, J., Zhao, J., 2013. ACOSampling: An ant colony optimization based undersampling method for classifying imbalanced DNA microarray data. *Neurocomputing* 101, 309–318.
- [105] Yun, J., Ha, J., Lee, J.S., 2016. Automatic determination of neighborhood size in SMOTE, in: *Proc. of the 10th International Conference on Ubiquitous Information Management and Communication*, pp. 100:1–100:8.
- [106] Zhang, H., Li, M., 2014. RWO-Sampling: A random walk over-sampling approach to imbalanced data classification. *Inf Fusion* 20, 99–116.
- [107] Zhang, H., Wang, Z., 2011a. A normal distribution-based oversampling approach to imbalanced data classification, in: *Advanced Data Mining and Applications*, Springer Berlin Heidelberg. pp. 83–96.
- [108] Zhang, L., Wang, W., 2011b. A re-sampling method for class imbalance learning with credit data, in: *International Conference of Information Technology, Computer Engineering and Management Sciences*, pp. 393–397.
- [109] Zhang, X., Li, Y., 2013. A positive-biased nearest neighbour algorithm for imbalanced classification, in: *Proc. of PAKDD 2013*, pp. 293–304.
- [110] Zhang, X.J., Tari, Z., Cheriet, M., 2017. KRNN: k Rare-class Nearest Neighbor classification. *Pattern Recognition* 62, 33–44.
- [111] Zhou, B., Yang, C., Guo, H., Hu, J., 2013. A quasi-linear SVM combined with assembled SMOTE for imbalanced data classification, in: *The 2013 International Joint Conference on Neural Networks (IJCNN)*, pp. 1–7.
- [112] Zieba, M., Tomczak, J.M., Gonczarek, A., 2015. RBM-SMOTE: Restricted boltzmann machines for synthetic minority oversampling technique, in: *Intelligent Information and Database Systems*, Springer International Publishing. pp. 377–386.